

2019 Goricon 풀이

2019. 09. 29

주최 : 경북대학교 알고리즘 문제해결 동아리 Gori



지원 : Startlink



출제 및 검수

annaria

djm03178

dogdriip

exqt

jame0313

스타후르츠

정답 : 18팀 (참가 팀 전부)

First Solve : 갈빙리타 (2 min)

Tag : 산수(?)

스타후르츠

T일 주기로 돈을 벌 수 있습니다.

단, 1일에는 수확을 할 수 없으므로 $\text{floor}((N-1) / T)$ 번 수확가능합니다.

답은 $\text{floor}((N-1) / T) * C * P$

클레어와 팰린드롬

정답 : 17 + ? 팀

First Solve : 갈빙리타 (13 min)

Tag : 문자열

클레어와 팰린드롬

ROTATOR

팰린드롬이 되려면 모든 i 에 대해 $S[i] = S[n-1-i]$ 를 만족해야합니다.

클레어와 팰린드롬

?O?A??R

$S[i]$ 가 ? 이면 $S[i] = S[n-1-i]$

둘 다 ? 이면 $S[i] = S[n-1-i] = \text{<아무 알파벳 소문자>}$ 로 교체하면 됩니다.

클레어와 팰린드롬

ROT?TOR

중앙에 ?가 있는 경우도 처리해야합니다.

제리와 톰 2

정답 : 14 + ? 팀

First Solve : 등수가 소수일 시 엄청난 환호를 받게될 팀 (62 min)

Tag : 분수

제리와 톰 2

이렇게 생겨먹은 분수를 연분수라고 합니다.

가장 아래에 있는 분수를 처리하면서 크기를 줄여나가면 됩니다.

분자와 분모를 최대공약수로 나누어서 출력해야 할거같지만 항상 최대공약수가 1이라 그냥 출력해도 됩니다.

수열과 시프트 퀴리

정답 : 9 + ? 팀

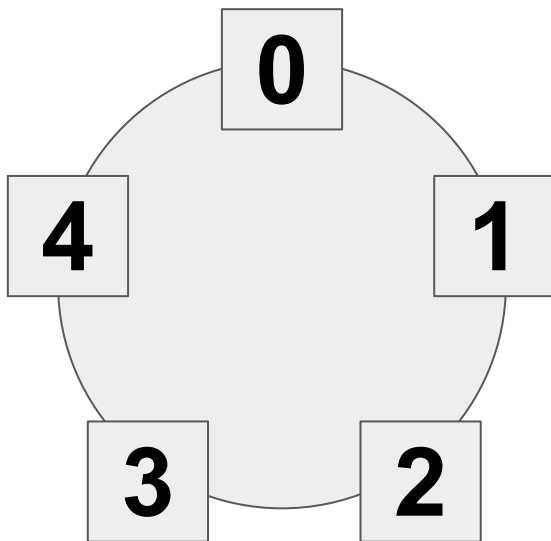
First Solve : 갈빙리타 (38 min)

Tag : 자료구조

수열과 시프트 쿼리

편의를 위해 1-index 에서 0-index로 바꾸고

배열을 선형이 아닌 원형이라고 생각해봅시다.



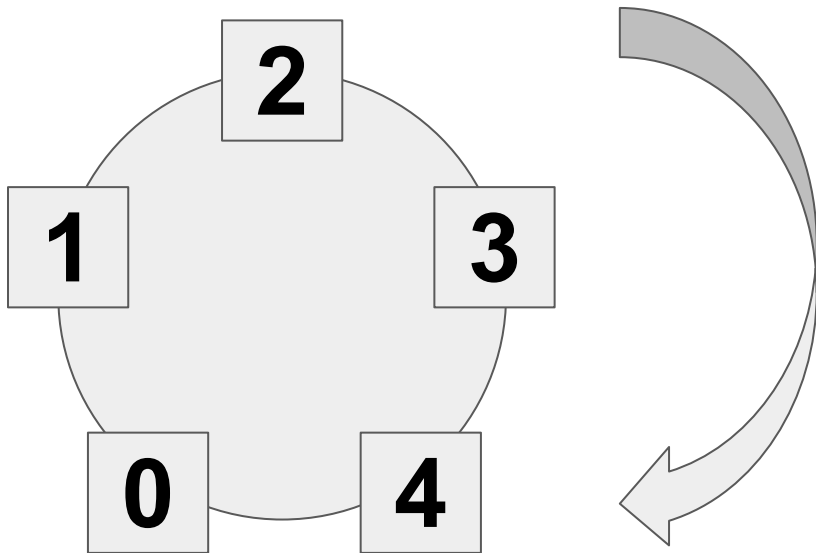
수열과 시프트 쿼리

수열을 쉬프트하면 원소들을 옮기지 말고 얼마나 돌아갔는지 기록합니다.

오른쪽으로 돌아간 칸의 수를 **OFFSET**이라고 합니다.

오른쪽으로 3칸 시프트

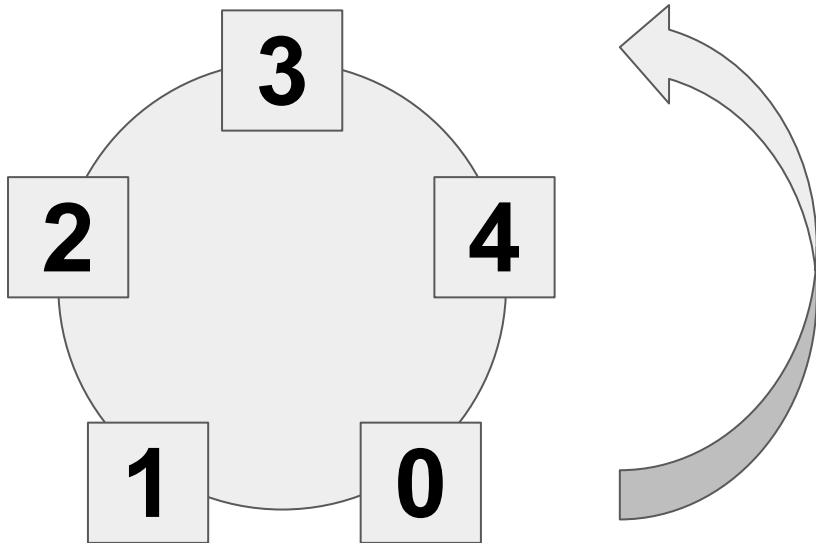
$$\text{OFFSET} = 0 + 3 = 3$$



수열과 시프트 쿼리

왼쪽으로 돌리는 건 빼주면 됩니다.

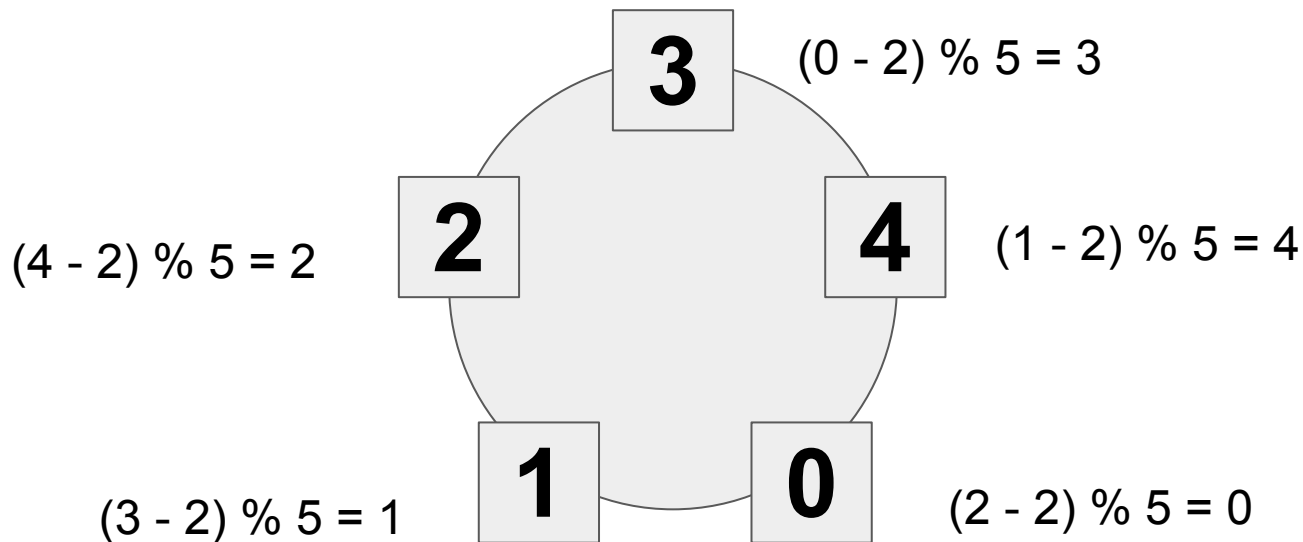
왼쪽으로 1칸 시프트
 $\text{OFFSET} = 3 - 1 = 2$



수열과 시프트 쿼리

OFFSET 만큼 돌아간 상태에서

i번째 원소의 위치는 $(i - \text{OFFSET}) \% n$ 입니다. (C/C++에선 나머지 음수 조심)



폴짝 게임

정답 : 3 + ? 팀

First Solve : 전현승 없는 전현승 팀 (53 min)

Tag : Dynamic Programming

수식 트리

정답 : 1 + ? 팀

First Solve : 전현승 없는 전현승 팀 (97 min)

Tag : 정렬, 트리

수식 트리

+와 -만 있어서 결과는 각각의 a_i 에 +1 또는 -1 를 곱한 것들의 합이 됩니다.

그럼 a_i 에 +1이 곱해질지 -1이 곱해질지 어떻게 알수있을까요?

수식 트리

루트 노드에서 출발하는데 초기에는
부호를 +1로 가지고 아래로 내려갑니다.

‘+’ 연산자 노드를 만나면 부호를 바꾸지
말고 자식들로 내려가고

‘-’ 연산자 노드를 만나면 오른쪽으로 갈
때만 부호를 바꾸어 내려갑니다.

이렇게 피연산자 노드에 도착하면 해당
노드에 곱해질 부호를 알 수 있습니다.

수식 트리

곱해질 '+' 부호와 '-' 부호의 개수 바뀌지 않으므로 큰 값들에게는 '+', 작은 값들에게는 '-' 를 붙여주는 것이 좋습니다.

[a1, a2, ..., an] 을 정렬한 후 부호들을 적절하게 곱한 합을 출력하면 답이 됩니다.

계산기

정답 : 2 + ? 팀

First Solve : 갈빙리타 (66 min)

Tag : 이진법, 비트연산

계산기

N을 2진수로 표현하면 접근이 쉽습니다.

12(10진법) -> 1100(2진법)

연산도 다음과 같이 생각할 수 있습니다.

[+] : 오른쪽에서 두번째 비트를 1로 바꾼다. 0100(2) -> 0110(2)

[-] : 필요 없습니다.

[*] : 2진수를 왼쪽으로 시프트 0110(2) -> 1100(2)

[/] : 2진수를 오른쪽으로 시프트 0110(2) -> 0011(2)

계산기

오른쪽에서 X번째 비트를 1로 만들고 싶으면

[+] 를 한번, 후에 [*] 를 X-2번하면 됩니다.

$$26(10) = 11010(2)$$

[+] [*] [+] [*] [*] [+]

2^4

2^3

2^1

계산기

홀수는?

앞 부분을 다 만든 후 $[*]$ $[/]$ 를 하면 1을 더할 수 있습니다.

$$27(10) = 11011(2)$$

$[+]$ $[*]$ $[+]$ $[*]$ $[*]$ $[+]$ $[*]$ $[/]$

2^4 2^3 2^1 2^0

계산기

제한이 2^{42} 미만이라 $[+]$ 와 $[*]$ 는 각각 최대 42번 등장하고 $[/]$ 는 최대 1번 등장합니다.

$42*2+1 = 85 < 99$ 이므로 넉넉하게 풀 수 있습니다.

맥주 축제

정답 : 2 + ? 팀

First Solve : ichigaya_arisa_loves_MintChocoLove (108 min)

Tag : 이분 탐색, 힙

맥주 축제

간의 레벨의 최솟값을 구하는 최적화 문제에서 간의 레벨을 고정하고 선호도의 합 M 을 채울 수 있는지 판단하는 결정 문제로 바뀌서 풀면 됩니다.

먼저 K 개의 맥주를 만족도를 오름차순으로 정렬을 해둡시다.

간의 레벨이 주어진다면 선호도가 가장 높은 N 개의 맥주를 쉽게 고를 수 있습니다.

간의 레벨과 최대 만족도의 합의 관계는 단조증가이므로 **Parametric Search**로 풀 수 있습니다.

구간 변수를 `int`로 잡으면 **overflow**가 일어나니 주의해야 합니다.

```
int MIDDLE = (RIGHT-LEFT)/2+LEFT; 또는 long long MIDDLE = (1LL*RIGHT+LEFT)/2;
```

시간 복잡도는 $O(N \log M)$

맥주 축제

레벨이 정해지면 레벨 이하의 맥주들을 모아 선호도가 가장 큰 **N** 개를 마시면 해당 레벨이 조건을 만족하는지 알 수 있습니다.

레벨 기준으로 정렬한 다음 **min-heap**으로 선호도를 관리하면서 합이 **M**이상이 되는 가장 작은 레벨을 찾을 수 있습니다.

시간 복잡도는 $O(K \log N)$

국경

정답 : 0 + ? 팀

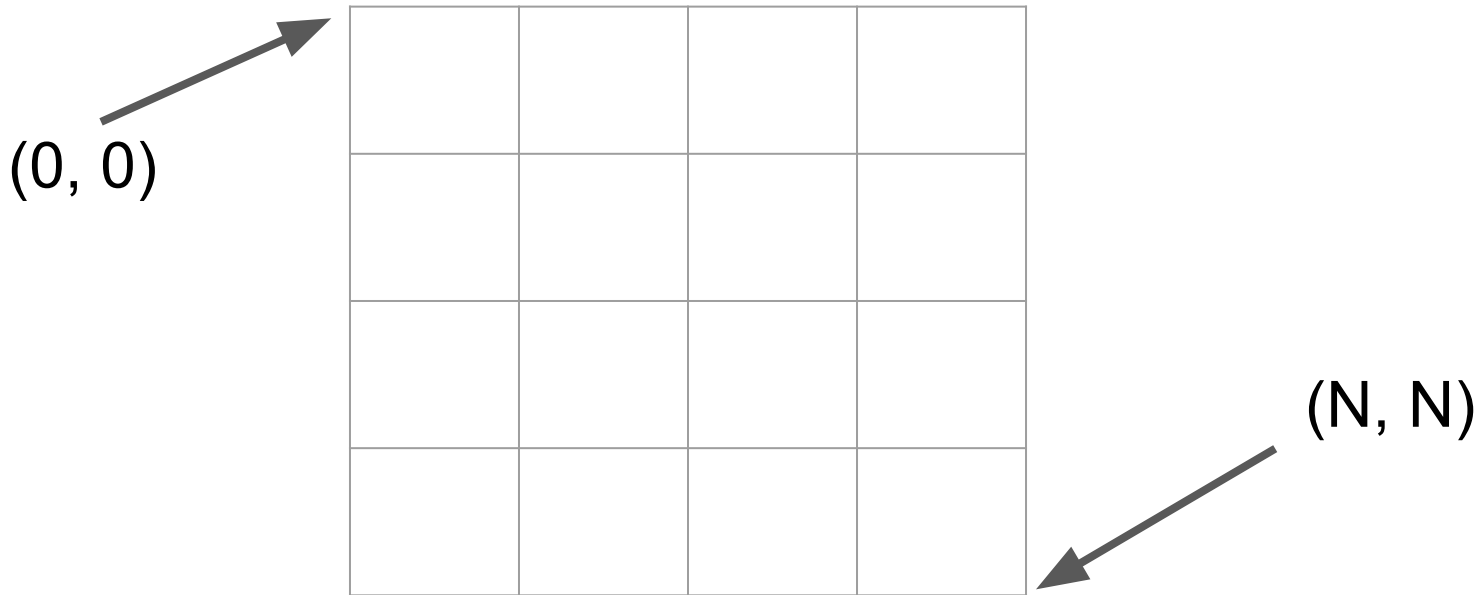
First Solve : ??? (? min)

Tag : 백트래킹, BFS/DFS, 구현

국경

왼쪽 상단이 좌표가 $(0, 0)$ 오른쪽 하단이 (N, N) 인 그리드를 만듭니다.

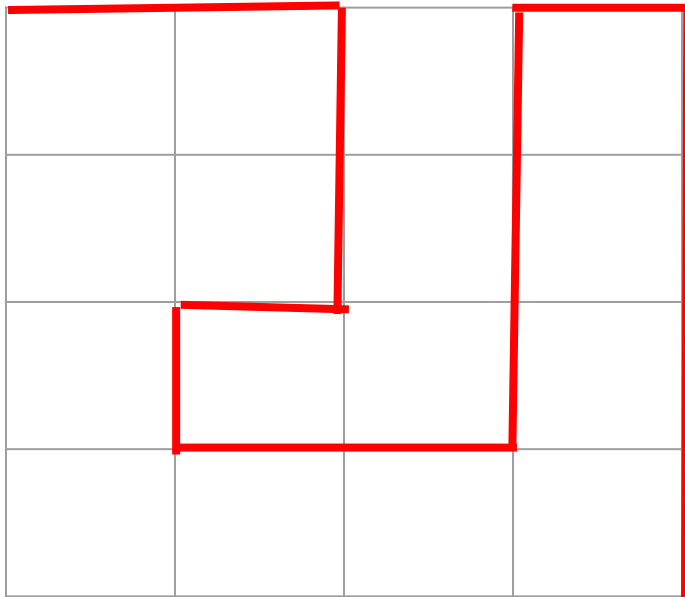
그래프의 정점을 칸이 아닌 꼭짓점으로 봅니다.



국경

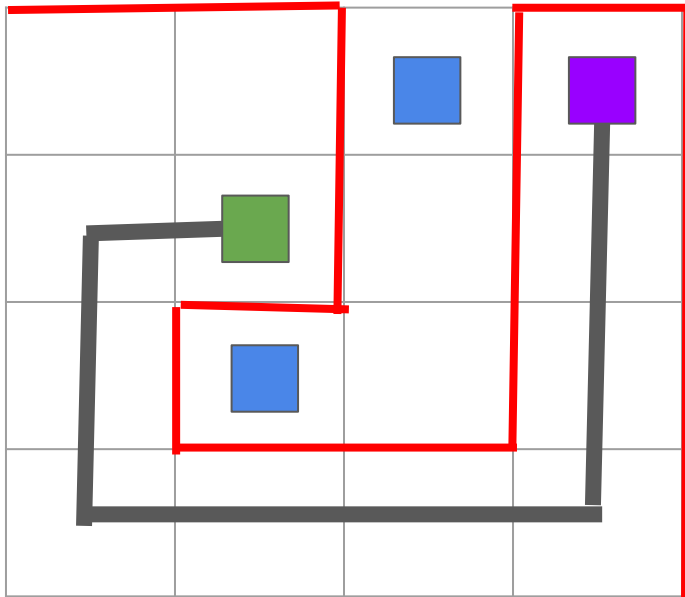
백트래킹으로 정점이 중복되지 않는 경로들을 찾아 그립니다.

(N=4 일때 정확히 8512개 있습니다.)



국경

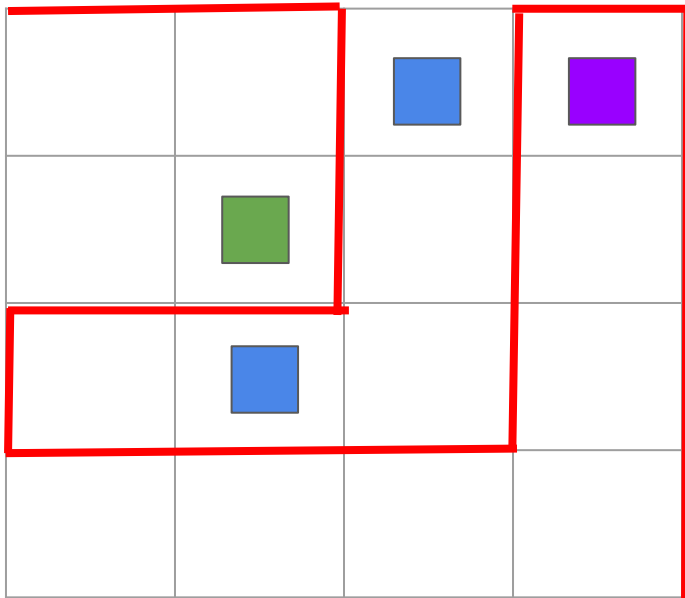
BFS 또는 DFS를 이용하여 아래와 같이 같은 영역에 있는데 색이 다른경우가 있는지 조사합니다.



국경

잘 나누는 경로를 찾았다면 문제에서 요구하는 형식으로 출력하면 됩니다.

구현하면 알아서 잘 해주시면 됩니다.



링고와 순열

정답 : 2 + ? 팀

First Solve : 갈빙리타 (46 min)

Tag :

링고와 순열

편의를 위해 $[1, 2, \dots, n]$ 를 $[0, 1, \dots, n-1]$ 로 교체합니다.

$[0, 1, \dots, n-1]$ 에서 임의의 수를 제일 앞으로 빼면 반전의 개수가 해당 수 만큼 증가합니다.

$[0, 1, 2, 3, 4]$ 0개

$[3, 0, 1, 2, 4]$ 3개

링고와 순열

한 개가 아닌 여러 개에도 적용됩니다.

단, 작은것부터 앞으로 옮겨야 합니다.

[3, 0, 1, 2, 4] 3개

[2, 3, 0, 1, 4] 4개

[2, 0, 1, 3, 4] 2개

[3, 2, 0, 1, 4] 5개

링고와 순열

그럼 이제 $[0, 1, 2, \dots, N-1]$ 에서 합이 K 가 되도록 몇개를 고르면 됩니다.

Dynamic Programming을 생각할 수는 있지만 제한이 너무 큽니다.

다음과 같은 Greedy한 방법을 생각해봅시다.

> 아직 사용하지 않은 수 중에서 가장 큰 수를 X 라고 하면

> $\min(K, X)$ 를 고르고 K 에서 그 만큼 빼는 과정을 반복합니다.

이 방법으로 항상 답을 찾을 수 있을까요?

링고와 순열

수학적 귀납법으로 증명해봅시다.

$P(N) = [1, 2, \dots, N]$ 에서 몇 개를 골라 합이 $0 \leq K \leq N(N+1)/2$ 를 만족하는 모든 K 를 되도록 만들 수 있다.

$P(1)$ 은 자명합니다. $K=0$ 이면 아무것도 고르지 말고 1이면 1을 고르면 됩니다.

$P(N-1)$ 이 참이면 이면 $P(N)$ 이 참임을 보여줍니다.

링고와 순열

$$K > N$$

$[1, 2, \dots, N-1]$ 에서 몇개를 골라 합이 $K-N$ ($0 \leq K-N \leq N(N-1)/2$) 이 되도록하는 방법이 존재하므로 여기에 N 이 붙여 합이 K 가 되도록 만들 수 있습니다.

$K-N \leq N(N-1)/2$ 는 $K \leq N(N-1)/2 + N (=N(N+1)/2)$ 이므로 $P(N-1)$ 이 참 일 때 $P(N)$ 도 항상 참입니다.

$$K \leq N$$

$[1, 2, \dots, N]$ 중에 K 가 존재하므로 K 하나만 고르면 됩니다.



GPSR



전현승 없는 전현승 팀

🏆 1등 🏆
갈빙리타